

Internship Task Outputs

1. Top 5 Highest Salary Employees

The screenshot shows the MySQL Workbench interface with a query window titled 'Query 1' containing the following SQL code:

```
29 SELECT *
30 FROM Employee
31 ORDER BY salary DESC
32 LIMIT 5;
33 SELECT dept_id,
34 COUNT(*) AS employee_count
35 FROM Employee
36 GROUP BY dept_id;
37 -- SELECT MAX(salary) AS second_highest_salary
```

The 'Result Grid' displays the results of the first query (SELECT * FROM Employee ORDER BY salary DESC LIMIT 5):

dept_id	employee_count
1	2
2	4
3	2
4	2

The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
14	10:36:28	SELECT e.emp_name, d.dept_name, e.salary FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id	10 row(s) returned	0.000 sec / 0.000 sec
15	10:36:58	SELECT e.emp_name, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id	10 row(s) returned	0.000 sec / 0.000 sec
16	10:37:14	SELECT dept_id, COUNT(*) AS total_employees FROM Employee GROUP BY dept_id HAVING COUNT(*) > 1	1 row(s) returned	0.016 sec / 0.000 sec
17	10:37:30	SELECT * FROM Employee WHERE hire_date >= CURRENT_DATE - INTERVAL 6 MONTH LIMIT 5	7 row(s) returned	0.016 sec / 0.000 sec
18	10:44:00	SELECT * FROM Employee ORDER BY salary DESC LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
19	10:44:00	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec

2. Department-wise Employee Count

The screenshot shows the MySQL Workbench interface with a query window titled 'Query 1' containing the following SQL code:

```
30 -- FROM Employee
31 -- ORDER BY salary DESC
32 -- LIMIT 5;
33 SELECT dept_id,
34 COUNT(*) AS employee_count
35 FROM Employee
36 GROUP BY dept_id;
37
```

The 'Result Grid' displays the results of the second query (SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id):

dept_id	employee_count
1	2
2	4
3	2
4	2

The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
15	10:36:58	SELECT e.emp_name, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id	10 row(s) returned	0.000 sec / 0.000 sec
16	10:37:14	SELECT dept_id, COUNT(*) AS total_employees FROM Employee GROUP BY dept_id HAVING COUNT(*) > 1	1 row(s) returned	0.016 sec / 0.000 sec
17	10:37:30	SELECT * FROM Employee WHERE hire_date >= CURRENT_DATE - INTERVAL 6 MONTH LIMIT 5	7 row(s) returned	0.016 sec / 0.000 sec
18	10:44:00	SELECT * FROM Employee ORDER BY salary DESC LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
19	10:44:00	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec
20	10:51:25	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec

3. Second Highest Salary

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left displays the 'data_science' schema. The 'Query' editor contains the following SQL code:

```
35 -- FROM Employee
36 -- GROUP BY dept_id;
37 SELECT MAX(salary) AS second_highest_salary
38 FROM Employee
39 WHERE salary < (
40 SELECT MAX(salary)
41 FROM Employee
42 );
```

The 'Result Grid' shows a single row with the value 75000.00 for the column 'second_highest_salary'. The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
16	10:37:14	SELECT dept_id, COUNT(*) AS total_employees FROM Employee GROUP BY dept_id HAVING...	1 row(s) returned	0.016 sec / 0.000 sec
17	10:37:30	SELECT * FROM Employee WHERE hire_date >= CURRENT_DATE - INTERVAL 6 MONTH L...	7 row(s) returned	0.016 sec / 0.000 sec
18	10:44:00	SELECT * FROM Employee ORDER BY salary DESC LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
19	10:44:00	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec
20	10:51:25	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec
21	10:53:12	SELECT MAX(salary) AS second_highest_salary FROM Employee WHERE salary < (SELECT ...	1 row(s) returned	0.016 sec / 0.000 sec

4. Employees Whose Salary is Greater Than Department Average

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left displays the 'data_science' schema. The 'Query' editor contains the following SQL code:

```
43 SELECT *
44 FROM Employee e
45 WHERE salary >
46 (
47 SELECT AVG(salary)
48 FROM Employee
49 WHERE dept_id = e.dept_id
50 );
```

The 'Result Grid' shows the following data:

emp_id	emp_name	salary	hire_date	dept_id
102	Neha	65000.00	2025-12-20	2
104	Priya	70000.00	2025-11-18	3
106	Sneha	80000.00	2025-10-10	2025-10-10
108	Pooja	75000.00	2026-04-22	4
109	Arjun	55000.00	2026-01-20	1

The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
17	10:37:30	SELECT * FROM Employee WHERE hire_date >= CURRENT_DATE - INTERVAL 6 MONTH L...	7 row(s) returned	0.016 sec / 0.000 sec
18	10:44:00	SELECT * FROM Employee ORDER BY salary DESC LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
19	10:44:00	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec
20	10:51:25	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec
21	10:53:12	SELECT MAX(salary) AS second_highest_salary FROM Employee WHERE salary < (SELECT ...	1 row(s) returned	0.016 sec / 0.000 sec
22	10:54:49	SELECT * FROM Employee e WHERE salary > (SELECT AVG(salary) FROM Employee WHER...	5 row(s) returned	0.000 sec / 0.000 sec

5. Inner Join Between Employee and Department Tables

The screenshot shows the MySQL Workbench interface with a query window open. The query is an inner join between the Employee and Department tables. The result grid shows the following data:

emp_name	dept_name	salary
Rahul	HR	45000.00
Arjun	HR	55000.00
Amit	IT	55000.00
Neha	IT	65000.00
Sneha	IT	80000.00

The output pane shows the execution of the query, with a message indicating that 10 rows were returned.

6. Left Join Between Employee and Department Tables

The screenshot shows the MySQL Workbench interface with a query window open. The query is a left join between the Employee and Department tables. The result grid shows the following data:

emp_name	dept_name
Amit	IT
Neha	IT
Rahul	HR
Priya	Finance
Rohan	Sales

The output pane shows the execution of the query, with a message indicating that 10 rows were returned.

7. GROUP BY with HAVING Clause

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
59 -- FROM Employee e
60 -- Open a script file in this editor
61 -- ON e.dept_id = d.dept_id;
62 • SELECT dept_id,
63 COUNT(*) AS total_employees
64 FROM Employee
65 GROUP BY dept_id
66 HAVING COUNT(*) > 2;
```

The Result Grid shows the following data:

dept_id	total_employees
2	4

The Output pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
20	10:51:25	SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id LIMIT 0...	4 row(s) returned	0.000 sec / 0.000 sec
21	10:53:12	SELECT MAX(salary) AS second_highest_salary FROM Employee WHERE salary < (SELECT ...	1 row(s) returned	0.016 sec / 0.000 sec
22	10:54:49	SELECT * FROM Employee e WHERE salary > (SELECT AVG(salary) FROM Employee WHER...	5 row(s) returned	0.000 sec / 0.000 sec
23	10:55:49	SELECT e.emp_name, d.dept_name, e.salary FROM Employee e INNER JOIN Department d O...	10 row(s) returned	0.000 sec / 0.000 sec
24	10:57:08	SELECT e.emp_name, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_j...	10 row(s) returned	0.000 sec / 0.000 sec
25	10:58:54	SELECT dept_id, COUNT(*) AS total_employees FROM Employee GROUP BY dept_id HAVING...	1 row(s) returned	0.000 sec / 0.000 sec

8. Employees Hired in the Last 6 Months

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
62 -- SELECT dept_id,
63 -- COUNT(*) AS total_employees
64 -- FROM Employee
65 -- GROUP BY dept_id
66 -- HAVING COUNT(*) > 2;
67 • SELECT *
68 FROM Employee
69 WHERE hire_date >= CURRENT_DATE - INTERVAL 6 MONTH;
```

The Result Grid shows the following data:

emp_id	emp_name	salary	hire_date	dept_id
101	Amit	55000.00	2026-01-15	2
103	Rahul	45000.00	2026-03-10	1
105	Rohan	50000.00	2026-05-01	4
107	Karan	60000.00	2026-02-12	3
108	Pooja	75000.00	2026-04-22	4

The Output pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
21	10:53:12	SELECT MAX(salary) AS second_highest_salary FROM Employee WHERE salary < (SELECT ...	1 row(s) returned	0.016 sec / 0.000 sec
22	10:54:49	SELECT * FROM Employee e WHERE salary > (SELECT AVG(salary) FROM Employee WHER...	5 row(s) returned	0.000 sec / 0.000 sec
23	10:55:49	SELECT e.emp_name, d.dept_name, e.salary FROM Employee e INNER JOIN Department d O...	10 row(s) returned	0.000 sec / 0.000 sec
24	10:57:08	SELECT e.emp_name, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_j...	10 row(s) returned	0.000 sec / 0.000 sec
25	10:58:54	SELECT dept_id, COUNT(*) AS total_employees FROM Employee GROUP BY dept_id HAVING...	1 row(s) returned	0.000 sec / 0.000 sec
26	10:59:42	SELECT * FROM Employee WHERE hire_date >= CURRENT_DATE - INTERVAL 6 MONTH L...	7 row(s) returned	0.000 sec / 0.000 sec

9. Find Duplicate Records

The screenshot shows MySQL Workbench with a query window titled 'SQL File 3'. The query is as follows:

```
70 SELECT emp_name,  
71 salary,  
72 COUNT(*) AS duplicate_count  
73 FROM Employee  
74 GROUP BY emp_name,  
75 salary  
76 HAVING COUNT(*) > 1;  
77
```

The Results Grid shows one result:

emp_name	salary	duplicate_count
Amit	55000.00	2

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
27	11:00:51	SELECT emp_name, salary, COUNT(*) AS duplicate_count FROM Employee GROUP BY emp_name, salary HAVING COUNT(*) > 1;	0 row(s) returned	0.000 sec / 0.000 sec
28	11:02:46	DELETE e1 FROM Employee e1 INNER JOIN Employee e2 ON e1.emp_id > e2.emp_id AND e1.emp_name = e2.emp_name AND e1.salary = e2.salary;	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE clause that would limit the rows to be updated.	0.000 sec
29	11:05:00	DELETE e1 FROM Employee e1 INNER JOIN Employee e2 ON e1.emp_id > e2.emp_id AND e1.emp_name = e2.emp_name AND e1.salary = e2.salary;	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE clause that would limit the rows to be updated.	0.000 sec
30	11:05:39	INSERT INTO Employee VALUES (111,'Amit',55000,'2026-01-15',2);	1 row(s) affected	0.016 sec
31	11:05:55	DELETE e1 FROM Employee e1 INNER JOIN Employee e2 ON e1.emp_id > e2.emp_id AND e1.emp_name = e2.emp_name AND e1.salary = e2.salary;	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE clause that would limit the rows to be updated.	0.016 sec
32	11:06:45	SELECT emp_name, salary, COUNT(*) AS duplicate_count FROM Employee GROUP BY emp_name, salary HAVING COUNT(*) > 1;	1 row(s) returned	0.016 sec / 0.000 sec

10. Remove Duplicate Records

The screenshot shows MySQL Workbench with a query window titled 'SQL File 3'. The query is as follows:

```
69 -- WHERE hire_date >= CURRENT_DATE - INTERVAL 6 MONTH;  
70 -- SELECT emp_name,  
71 -- salary,  
72 -- COUNT(*) AS duplicate_count  
73 -- FROM Employee  
74 -- GROUP BY emp_name,  
75 -- salary  
76 -- HAVING COUNT(*) > 1;  
77 -- SET SQL_SAFE_UPDATES = 0;  
78 DELETE e1  
79 FROM Employee e1  
80 INNER JOIN Employee e2  
81 ON e1.emp_id > e2.emp_id  
82 AND e1.emp_name = e2.emp_name  
83 AND e1.salary = e2.salary;
```

The Results Grid is empty.

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
29	11:05:00	DELETE e1 FROM Employee e1 INNER JOIN Employee e2 ON e1.emp_id > e2.emp_id AND e1.emp_name = e2.emp_name AND e1.salary = e2.salary;	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE clause that would limit the rows to be updated.	0.000 sec
30	11:05:39	INSERT INTO Employee VALUES (111,'Amit',55000,'2026-01-15',2);	1 row(s) affected	0.016 sec
31	11:05:55	DELETE e1 FROM Employee e1 INNER JOIN Employee e2 ON e1.emp_id > e2.emp_id AND e1.emp_name = e2.emp_name AND e1.salary = e2.salary;	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE clause that would limit the rows to be updated.	0.016 sec
32	11:06:45	SELECT emp_name, salary, COUNT(*) AS duplicate_count FROM Employee GROUP BY emp_name, salary HAVING COUNT(*) > 1;	1 row(s) returned	0.016 sec / 0.000 sec
33	11:10:20	SET SQL_SAFE_UPDATES = 0;	0 row(s) affected	0.000 sec
34	11:10:37	DELETE e1 FROM Employee e1 INNER JOIN Employee e2 ON e1.emp_id > e2.emp_id AND e1.emp_name = e2.emp_name AND e1.salary = e2.salary;	1 row(s) affected	0.015 sec